The Map of Lean 4

v16.4 · Research group meeting · 29 October 2026

For this room: PDE is home turf, Lean is not. No computer science assumed.

First: what Lean already knows.
Then: how to name it and build on it.
Along the way: where each word lives (A/B/C/D).

Open Lean4_practice.lean alongside this deck. Type as we go.

We end on the gap: the analysis layer is there; the PDE layer is not.

The premise: mathematics is larger, Lean checks a fragment

- Mathematics: ideas, definitions, examples, proofs in ordinary language.
- Lean 4: a formal language. A small kernel checks that a term has the stated type.
- Lean does NOT decide whether your formalization matches the math you meant.
- Start from the mathematics. Use Lean to make one piece checkable.

Three ideas to keep in mind

Curry–Howard

Propositions are types. Proofs are terms of those types.

Meaning triangle

A symbol never touches its referent directly. Thought mediates.

Desirable difficulty

Retrieval and spacing help. You will get confused on purpose.

What is computation? Two answers, one result

Turing (1936)

An infinite tape divided into cells.
A read/write head moves left and right.
At each step: read the cell, decide
what to write, move, change state.

That is the whole machine.

Church (1936)

```
No tape, no states. Everything is a function.
Three rules only:
  x        a variable
  λx. e    a function (input x, body e)
  f a      apply f to a

That is the whole calculus.
```

They proved these two models have the same power. This equivalence is a pillar of the Church-Turing thesis. Lean's kernel is a typed lambda calculus.

What is a proof? From Hilbert to machine-checking

- Hilbert (1900): write all mathematics as axioms + fixed inference rules. Then a proof is mechanical — no intuition needed.
- Gödel (1931): no system strong enough for arithmetic can prove its own consistency. But this does NOT kill the project — it says you cannot do it from inside.
- What survives: a single proof can still be written so a machine checks every step. That is what Lean does.

What is a proof, concretely?

- Start from axioms. Apply inference rules one at a time.
- The most basic rule: modus ponens — if you have P→Q and P, you may conclude Q.
- Every step is mechanical. A machine can verify that no step was skipped.

Curry-Howard (1): propositions are types, proofs are terms

1. **A proposition P** is **a type** (Prop is the bottom level of Lean's universe)
2. **A proof of P** is **a term** of that type
3. **So P → Q** is **the function type**, and its proof is a **function**: takes a proof of P, returns a proof of Q

```
-- Logic: "P implies Q" means you can build Q from P.
-- Program: a function P → Q does exactly that.

theorem modus_ponens (P Q : Prop) : (P → Q) → P → Q := by
  intro f p  -- assume f : P → Q, p : P
  exact f p  -- apply the method to the proof
```

Modus ponens (logic): from P→Q and P, conclude Q.
Function application (program): from f : P→Q and p : P, get f p : Q.
Same rule. Same shape. Lean sees them as ONE thing.

Curry-Howard (2): logic / type / category — one table

Logic	Type theory	Category theory	Why
`P ∧ Q`	`P × Q`	product	need BOTH proofs
`P ∨ Q`	`P ⊕ Q`	coproduct	need EITHER proof
`P → Q`	`P → Q`	exponential	function: P→Q
`¬P`	`P → False`	function to empty	P implies False
`P ↔ Q`	`Iff P Q`	hom-set	both directions
`∀ x, P(x)`	`Π x, P x`	dependent prod	function: x→proof
`∃ x, P(x)`	`Σ x, P x`	dependent sum	pair: x + proof
`x = y`	`Eq x y`	identity type	proof of equality
`⊤`	`Unit/True`	terminal obj	one trivial proof
`⊥`	`Empty/False`	initial obj	no proof at all

Three names for one thing. ¬P = P→False. P↔Q = Iff (a structure holding two functions). x=y = Eq x y.

Notation: logic books write implication with one arrow (→). Type-theory texts also use a double arrow (⇒) to tell the two readings of A→B apart. Lean has only → — the function type and the implication are the same thing.

Curry-Howard (2b): why ∀ is a function, ∃ is a pair

∀ x, P(x) — give me x, I return a proof

```
-- "for all n, 0+n = n" means:
-- give me any n, I give you a proof

example : (n : Nat) → 0 + n = n := by
  intro n
  induction n with
  | zero => rfl
  | succ k ih => rw [Nat.add_succ, ih]

-- ↑ this IS a function.
-- ↑ n is the input, the body returns the proof.
```

∃ x, P(x) — I hand you x AND a proof

```
-- "there exists k such that 4 = 2*k"
-- means: I give you k=2, AND a proof

example : ∃ k : Nat, 4 = 2 * k := by
  refine ⟨2, ?_⟩
  rfl

-- ↑ refine ⟨2, ?_⟩: give the pair, leave me a hole
-- ↑ ⟨2, rfl⟩ is a PAIR:
--   first field  = 2  (the witness)
--   second field = proof that 4=2*2
```

P(x) varies with x: P(0) = "0=0" (has proof), P(1) = "1=0" (no proof).
Π x, P(x) = a function that gives a proof for EVERY x. Σ x, P(x) = one specific x + its proof.
∀ = Π = function. ∃ = Σ = pair. Lean writes ∀ as (x : α) → P x, ∃ as ⟨witness, proof⟩.

Curry-Howard (3): proof = program, checked = compiled

- Proving a theorem = writing a function whose type is the theorem statement.
- Checking a proof = the type checker verifying that function.
- Simplifying a proof = running / evaluating the program.

```
example : 2 + 2 = 4 := by
  rfl
--   ↑ this is a proof.
--   It is also a term of type (2 + 2 = 4).
--   rfl checks that 2+2 and 4 reduce to the same thing.
--   That is β/iota reduction doing its job.
```

Lean's kernel does one thing: check types. It has no idea it is also checking proofs.

What IS a type? Four rules — not a set

A type is NOT a collection of elements. It is COMPLETELY DEFINED by four rules. Example: the function type A → B.

1. Formation	`If A : Type and B : Type, then A → B : Type.`	When is this a valid type?
2. Introduction	`If x : A gives t : B, then (fun x => t) : A → B.`	How do you BUILD a term? (lambda / constructor)
3. Elimination	`If f : A → B and a : A, then f a : B.`	How do you USE a term? (application / match)
4. Computation	`(fun x => t) a  reduces to  t[a/x].`	What happens when intro meets elim? (beta reduction)

No analogy. No 'set of functions'. Every type is defined by four rules — formation, introduction, elimination, computation. Same four slots every time, different content: inductives use constructors and recursors (next slide); Empty has zero constructors, so no introduction rule at all.

The four rules, in Lean

Formation	`you write  inductive / def / structure`	you declare: this is a type now
Introduction	`constructors:  Nat.zero, Nat.succ, Or.inl`	these are the ONLY ways to make a term
Elimination	`match / recursor:  Nat.rec, Or.casesOn`	this is the ONLY way to USE a term
Computation	`kernel reduces automatically;  rfl  checks it`	intro meets elim → simplification

When you write inductive Nat where | zero | succ
you give the INTRODUCTION rules. Lean auto-generates the ELIMINATOR (Nat.rec) and COMPUTATION rules.
This is why inductive is the single most powerful keyword in Lean:
one declaration pins down all four rules — the keyword gives FORMATION,
you write INTRODUCTION, Lean generates ELIMINATION and COMPUTATION.

Precision: introduction & computation are per-constructor; formation & elimination are per-type (one recursor).
The rules for Type u itself — variables, conversion, universes — are the same four rules one level up. You never declare those.

The glue: syntax symbols — not a home, but you see them everywhere

Before the four homes: these symbols themselves are not terms — but expressions built from them DO have types.

Four bracket types:

`( e )`	grouping, function application: f (x+1)
`{x : α}`	implicit arg — caller omits, Lean infers
`[inst : C]`	instance arg — Lean searches automatically
`⟨a, b⟩`	anonymous constructor — build a pair/struct value

Other glue:

`:`	has type: n : Nat (left of : is a term, right is its type)
`:=`	defined as: def x := 5 (the body equals the name)
`→`	arrow: Nat → Nat (also written as function type)
`_`	placeholder: Lean figures it out for you
`|`	case separator: inductive constructors, match arms
`·`	focus block: in by-mode, start proving the current goal
`#`	command: #check, #eval — ask Lean a question

Every word lives in one of four homes

A Environment

You ask Lean about the room.
#check, #eval, import, open, namespace.

B Declaration

You add something new to the room.
def, inductive, structure, theorem.

C Term / Type

An expression itself.
2, fun n => n+1, Color.red, match ...

D Proof strategy

A tactic that helps build a C term.
by intro n; rw [h]; rfl.

How to read this deck: look at the screen first, get names later

1	**See**	Open Lean. It already knows numbers, booleans, strings. #check them.
2	**Build**	Give things names with def. Build new types with inductive.
3	**Prove**	A proof is just a term of a proposition type. Tactics help build it.

Coming from PDE? Slide 33 — the last one — tells you where Mathlib actually stands for analysis.

You do not need any computer science to follow the arc. Everything today happens on one screen, and every line you see compiles.

Open Lean. The room is already full.

Lean already knows all of these. You did not declare any of them. Type them and watch the right panel.

```
#check 2            -- Nat
#check true         -- Bool
#check "hello"      -- String
#check [1, 2, 3]    -- List Nat
#check (2, true)    -- Nat × Bool
#check Nat          -- Type
#check Type         -- Type 1
```

- #check asks: what type is this?
- The panel on the right (infoview) answers.
- 2 is not 'an integer'. It is a term of type Nat.
- Nat itself has a type: Type.
- Type itself has a type: Type 1.

Do not memorize Type 1 yet.
Just notice: types themselves have types.

A type classifies a term

Read ":" as has type. Left of : is a term. Right of : is what kind of term it is.

```
#check (2 : Nat)
#check (true : Bool)
#check (fun n => n + 1 : Nat → Nat)
#check (2 + 2 = 4)   -- this is a Prop
```

- Nat, Bool carry data you can compute with.
- Prop carries propositions.
- A term of type P ∧ Q is a proof of P ∧ Q.
- Most proofs cannot be taken apart to compute data.

Why this comes first: a theorem statement IS a type. A proof IS a term of that type. Everything later hangs off this one sentence.

A · Environment · Ask Lean four questions

```
#check (2 + 2)        -- Nat
#eval  (2 + 2)        -- 4
#reduce (1 + 2)       -- 3
-- #print Nat.add     -- see definition
```

`#check`	What type is this expression?
`#eval`	Run it and print the value.
`#reduce`	Fully simplify, show the result.
`#print`	Show how something is defined.

These four commands only READ the environment. They never create anything.

A · Environment · Organize names

```
-- OPTIONAL: this one line needs Mathlib installed.
-- Delete it and everything else below still works.
import Mathlib.Tactic   -- bring outside names in

namespace Lesson         -- give names an address
def two : Nat := 2
end Lesson

#check Lesson.two

-- scoped open: only affects the next line
open Lesson in
#check two
```

- import: make names from other files available.
- namespace: a folder for names, closed by end.
- open: skip the folder prefix.
- section / variable: temporary assumptions, vanish at end.

You rarely write these by hand in a day. You need them because Lean's standard library puts everything in folders.
The import line is the only thing on this slide that needs Mathlib. Plain Lean (Init) is enough for everything else in this deck.

B · Declarations · Name a term, step by step

```
-- Step 1: give a value a name
def two : Nat := 2

-- Step 2: a function (input -> output)
def double (n : Nat) : Nat := n + n

-- Step 3: use them
#eval double two     -- 4

-- Step 4: a proposition is a type. rfl proves it.
example : 2 + 2 = 4 := rfl

-- Step 5: name the theorem (same as example, but reusable)
theorem two_plus_two : 2 + 2 = 4 := rfl
```

- def: name a value.
- example: check, do not store.
- theorem / lemma: name a proof.
- axiom: not shown on purpose.

axiom means: declare a name and a type, but give no definition.
Skip it for now. It is how you "cheat" — you do not need to cheat yet.

Two buckets: what Lean gives you, what you build

Kernel: three type-formers

The kernel only knows three ways to form a type:

```
Sort   (Prop, Type, Type 1 ...)
→ / ∀  (Pi type)
Quot   (quotient)
```

These are hard-coded. You never declare them.
This is a mechanism map, not a closed inventory — reduction and recursion also live here.

You build with inductive

`inductive` → the main mechanism
`Your Color, MyNat, Point ...`
`AND Nat, Bool, List!`
(they live in the Init library, auto-loaded at startup, but they are written exactly like your Color; String is a structure — see below)

`structure` → inductive, one constructor
`class` → structure + searchable
`abbrev` → just a name, not a type

Why this matters for this room

These are not two languages. There is exactly ONE way to add a type, and it is **`inductive`** — structure, class and abbrev are all inductive in disguise (slides 22 to 24).

Every type in Mathlib — Measure, Lp, Distribution, the Sobolev space — was built that way. Learning Lean is not learning a new branch of mathematics; it is learning one keyword that happens to scale to 8,624 files.

B · inductive · finite choices (an enum)

```
inductive Color where
  | red
  | green
  | blue
```

Now ask what you made:

```
#check Color.red   -- Color
#check Color       -- Type
```

Use it — pattern match ALL cases:

```
def isWarm (c : Color) : Bool :=
  match c with
  | .red   => true
  | .green => false
  | .blue  => false
```

```
#eval isWarm Color.red    -- true
```

Note: the match covers all three.
Miss one and Lean tells you.

B · inductive · a recursive type (this is how numbers work)

```
inductive MyNat where
  | zero
  | succ (n : MyNat)
```

- zero : no input, gives one MyNat.
- succ : takes a MyNat, returns a bigger MyNat.
- Every MyNat is either zero, or succ of a smaller one.
- That recursion is exactly what induction proofs follow.

Build values from the constructors:

```
#check MyNat.zero           -- MyNat
#check MyNat.succ           -- MyNat → MyNat
#check MyNat.succ MyNat.zero -- MyNat ("one")

def one  := MyNat.succ MyNat.zero
def two' := MyNat.succ one
```

Lean's built-in Nat is defined almost exactly this way.
You just rebuilt the natural numbers.

B · structure · one constructor, named fields

```
structure Point where
  x : Nat
  y : Nat
```

This is shorthand for:

```
-- inductive Point where
--   | mk (x : Nat) (y : Nat) : Point
--
-- one constructor; its arguments ARE the fields
```

Two ways to build the same value:

```
def p1 : Point := { x := 3, y := 4 }
def p2 : Point := Point.mk 3 4
-- p1 and p2 are the SAME value.
```

Build and read:

```
def p : Point := { x := 3, y := 4 }

#eval p.x    -- 3
#eval p.y    -- 4
```

structure gives you named fields
and .x / .y projections for free.

It is not a new mechanism.
It is inductive with exactly one
constructor + a name for each field.

In Mathlib, Measure is a structure.

B · class / instance · a structure Lean can search for

```
-- named MyAdd because Add already exists in the library
class MyAdd (α : Type) where
  myAdd : α → α → α

instance : MyAdd Nat where
  myAdd n m := n + m
```

- class: declares an interface.
- instance: provides one implementation.
- Lean searches for it automatically.
- The real + on Nat works the same way: it finds Add Nat.

You almost never write classes by hand. The point is: class and instance are not new mechanisms. class is a structure marked as searchable. instance is just a value of that structure type. Everything reduces back to inductive.

B · abbrev · a nickname, not a new type

```
abbrev Pair := Nat × Nat

def myPair : Pair := (3, 4)
```

- Pair IS Nat × Nat.
- It does not create a new type.
- It just makes the source shorter.
- Contrast: inductive really does add a new type.

```
So the hierarchy of "making a type" is:

  inductive  →  genuinely new type, new constructors
  structure  →  inductive with one constructor + named fields
  class      →  structure + searchability
  abbrev     →  not a new type at all, just a name
```

C · The universe staircase

```
#check (2 + 2 = 4)   -- Prop     (Sort 0)
#check 2              -- Nat
#check Nat            -- Type     (Sort 1)
#check Type           -- Type 1   (Sort 2)
```

- Prop: propositions. Proofs live here.
- Type: ordinary data types.
- Type 1: the type of Type.
- In the same spirit as Grothendieck universes.
- No 'set of all sets' paradox — you just climb up.

Before we look at terms: types themselves live on a staircase. Prop at the bottom, Type above it, Type 1 above that — forever upward.

C · Terms · five forms you have already used

```
-- 1. a name
#check two

-- 2. application (supply an input)
#check double two

-- 3. a lambda (build a function)
#check (fun n => n + 1 : Nat → Nat)

-- 4. a constructor (build a value the declared way)
#check Color.green

-- 5. a match (read by constructors)
def describe (c : Color) : String :=
  match c with
  | .red   => "warm"
  | .green => "ok"
  | .blue  => "cold"
```

- Each form has its own shape.
- two and double two do not look alike.
- match must cover every constructor.
- This is the vocabulary of expressions.

C · Three tools that look similar but do different jobs

```
-- let: name a local VALUE inside a term
def f : Nat :=
  let y := 6
  y + 1
-- #eval f  => 7

-- have: name a local FACT inside a proof
example : 0 = 0 := by
  have h : 0 = 0 := rfl
  exact h

-- .1: read one field of a pair
def g : Nat :=
  let p := (3, true)
  p.1
```

- let lives in term mode.
- have lives in proof mode (by ...).
- .1 / .2 read pair fields.
- Mixing them up is common. Notice where each is allowed.

C · → and ∀ are the same thing (Pi type)

```
-- Non-dependent: result does NOT mention the input
#check (Nat → Nat)

-- Dependent: result DOES mention n
#check ((n : Nat) → 0 + n = n)
```

Lean prints the dependent arrow as ∀ — they are the same thing:

```
-- Lean's output for #check above (Mathlib imported):
--   ℕ → ℕ : Type
--   ∀ (n : ℕ), 0 + n = n : Prop
```

The lambda calculus is the math underneath: fun n => ... IS a lambda. => is the separator inside fun / match; ← is the rewrite-direction marker; ↔ (also written <->) is Iff — a structure, so it is a type too; only → doubles as a function type.

C · A constructor is one allowed way to build a term

`Color.red`	no input	one of three choices
`MyNat.succ n`	one input	build a bigger MyNat
`And.intro hp hq`	two proofs	build P ∧ Q
`⟨w, h⟩`	witness+proof	build ∃ k, P k
`rfl`	no input	Eq.refl when sides match

The constructors of a type are exactly the ways Lean lets you build it. match reads them back.

One inductive, three ways you touch it

B Declare (slide 21)

```
-- inductive MyNat where   (already declared on slide 21)
--   | zero
--   | succ (n : MyNat)
-- You do not re-declare it here.
```

C Build / read

```
#check MyNat.succ MyNat.zero

-- match reads one constructor at a time
-- (see describe in practice file)
```

D Prove (shape only)

```
-- (shape only — not runnable here)
-- induction n with
-- | zero   => rfl
-- | succ k ih =>
--   rw [...]  -- see slide 32 for the
--             -- fully worked example
```

induction follows the constructors.
zero case, then succ case with an
induction hypothesis ih.

D · Proof strategies · tactics help you build a C term

`intro n`	Assume n. Starts a function term.
`exact h`	Hand over h. h must match the goal's type.
`rfl`	Close a definitional equality.
`rw [h]`	Rewrite with h. Leaves a new goal.
`sorry`	Leave an explicit gap. Not a proof.

Tactics are not the stored proof. The kernel stores the resulting C term. Tactics are a meta-program that helps you write that term.

D · One complete proof, every step visible

```
theorem nat_zero_add (n : Nat) : 0 + n = n := by
  -- goal: 0 + n = n
  induction n with
  | zero =>
    -- goal: 0 + 0 = 0
    rfl
  | succ k ih =>
    -- ih : 0 + k = k
    -- goal: 0 + succ k = succ k
    rw [Nat.add_succ]
    -- goal: succ (0 + k) = succ k
    rw [ih]
    -- goal: succ k = succ k (rw closes it)
-- note: core already has Nat.zero_add;
-- we rename it to avoid a name collision.
```

- induction splits on the constructors.
- zero case closes by rfl.
- succ case: two rewrites; the second closes it.
- Watch the infoview after each line.

Where the analysis stands in Lean today (measured, not guessed)

Lean 4.35.0 + Mathlib, 8,624 .lean files. Every green line was #check-ed on this machine; every red line appears in no definition or theorem.

Already in Mathlib

- ✓ **Measure & integration**
  `MeasureTheory.Measure, .integral, .Integrable`
- ✓ **Lp spaces**
  `MeasureTheory.Lp`
- ✓ **Distributions, tests**
  `Distribution, TestFunction, SchwartzMap`
- ✓ **Sobolev space (a type)**
  `BesselPotentialSpace = H^{s, p}(E,F)`
- ✓ **Sobolev inequality**
  `MeasureTheory.eLpNorm_le_eLpNorm_fderiv`
- ✓ **Laplacian, harmonic fns**
  `Laplacian.laplacian, InnerProductSpace.HarmonicAt`
- ✓ **Gronwall, Fourier**
  `gronwallBound, IsPicardLindelof, FourierTransform`

Note: "Δ" is a scoped notation — open it first, or type the full name.

Not there (grep over all 8,624 files)

- ✗ **curl** `0 code hits`
- ✗ **mollifiers / smoothing** `0 code hits`
- ✗ **vorticity, transport eq.** `0 code hits`
- ✗ **Euler equations** `0 code hits`
- ✗ **weak solution spaces** `0 code hits`
- ✗ **Navier-Stokes** `0 code hits`
- ✗ **Stokes' theorem** `1 hit, in a docstring`
- ✗ **a PDE library** `no Analysis/PDE dir`

"code hits" = definitions and theorems, not docstrings.
Stray hits: "curl" (an HTTP-tool docstring), "smoothing" (Akra–Bazzi notes).

Mathlib has the Laplacian, but not **curl** — the analysis layer is there; the PDE layer is not. That gap is where this group's expertise lives.